

Intro to Backend and APIs

Day 3

Prerequisites

- [Python](#) 3.11 or newer
- [Postman](#) installed on the system
- Clone or download the repo github.com/mukesharyal/intro-to-backend

Iceberg of Web Development

**VISIBLE LAYER:
FRONTEND**
(WHAT USERS SEE)



**HIDDEN LAYER:
BACKEND &
INFRASTRUCTURE**
(HOW IT WORKS)

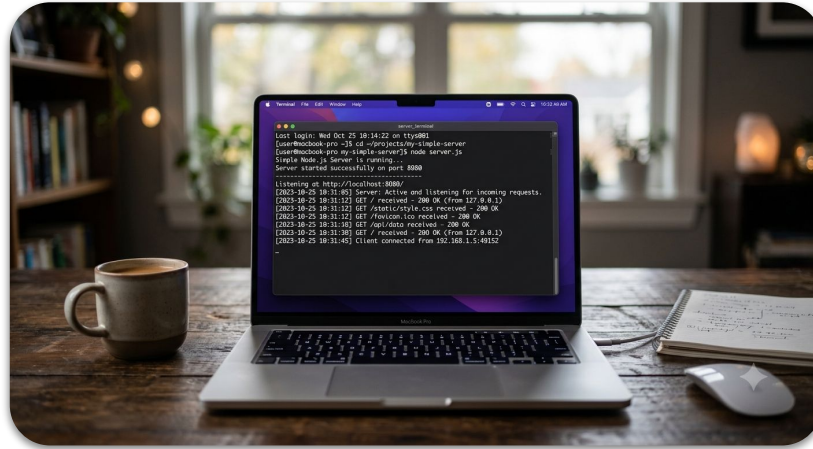


Objectives

- To make backend less **intimidating**
- To understand how systems (computers) **connect** and **communicate** with each other
- To develop an **interface** for systems to communicate

So what IS a backend?

- A backend is simply a **computer**



But is it *any* computer?

- It **can** be, but that wouldn't be **helpful**
- It has *similar* OS, CPU architecture, memory, and network interface
- Has maybe **64** CPUs instead of one
- Has not GBs, but **TBs** of RAM
- Has **dozens** of network interfaces
- Uses **virtualization** to efficiently use the resources

But is it *any* computer?

- We use a **portion** of the resources for hosting our **assets**

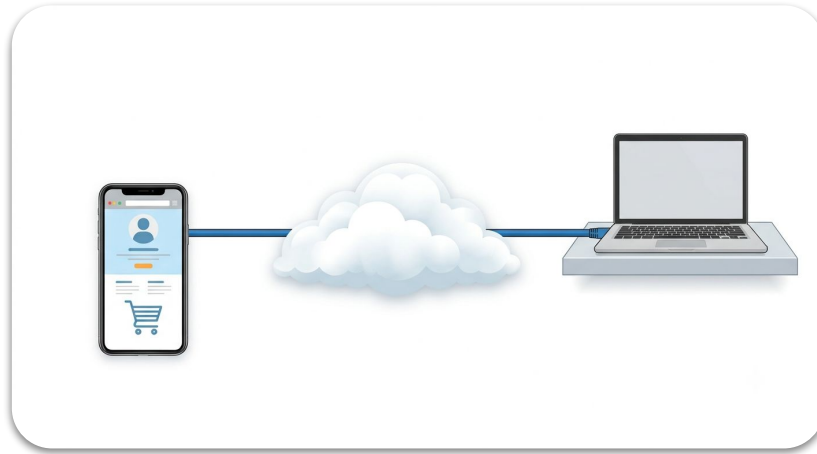


Requirements for a backend

- It needs to be **accessible** from the **internet**
- It needs to be up **24/7**
- It should accept **requests** from any computer and give some **response**
- ...which means if our **laptop** is also able to **fulfill** these **requirements**, it can become a **backend system**

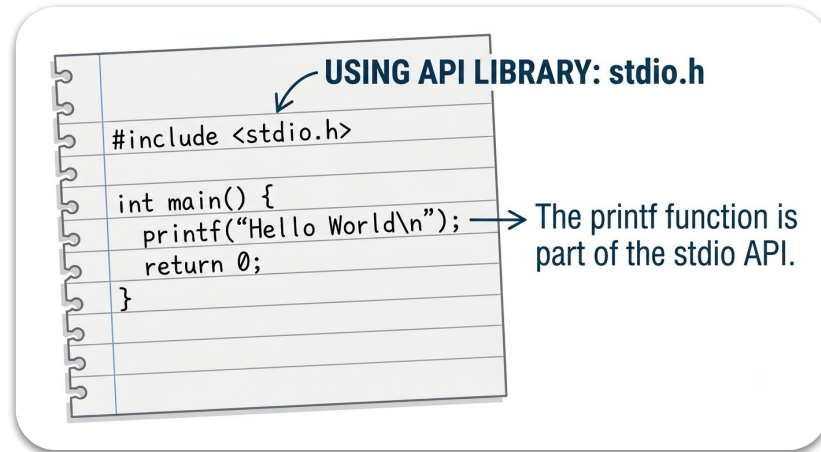
Conceptually...

- A client **requests** to our system, and it **responds** over the **internet**



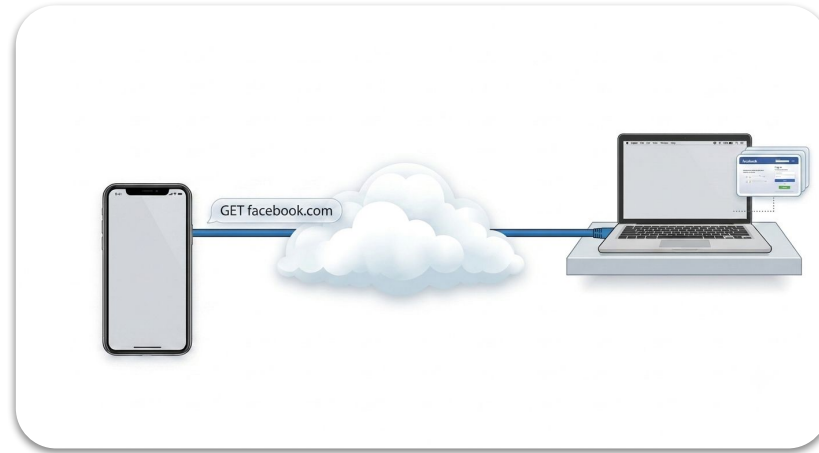
How does the client “request” the server?

- Via **APIs: Application Programming Interface**
- Essentially, a **function** or an **address** which belongs to the **system**



API over the internet

- Accessing a **website** from the **browser**



Designing APIs

- Suppose we are working with **users** and their **details**
- For **getting** the users' details, say we have an endpoint **/get-users**
- For **storing** a new user's details, say we have an endpoint **/add-user**
- When the complexity of our system increases, keeping track of the endpoints becomes difficult

Designing APIs

- We will need some **conventions** to help us with this
- **REST** (**R**epresentational **S**tate **T**ransfer) API is such a set of conventions
- There are just **2 principles**:
 - The **URL** (endpoint) represents the **resource**
 - The **HTTP method** represents the **action**

The URL represents the resource

- We are working with **users** in **both** the cases
- For getting the users' details, the endpoint would be **/users**
- For storing a new user's details, the endpoint would be **/users**

- Wait what..? Aren't they the same???

The HTTP method represents the action

- Even though the **URL** is the **same**, the **HTTP method** would be **different**
- For getting the users' details, the endpoint would be **/users** with **GET**
- For storing a new user's details, the endpoint would be **/users** with **POST**

- Because the **resource** is **users** in **both** cases, but **actions** are **different**

HTTP Methods

- When we send a **request**, the address **isn't** the only information sent
- We also **specify** various HTTP methods
- For example, when we type something into a browser address bar, we are actually sending a **GET** request to the server

Summary

URL: https://www.google.com/
Status: 200
Source: Network
Address: 2001:4860:482d:7700::443

Request

GET / HTTP/1.1
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-GB,en-US;q=0.9,en;q=0.8

HTTP Methods

- Some common HTTP methods include:
- **GET**: Used to query some data
- **POST**: Used to send some data
- **PUT**: Used to replace some data
- **DELETE**: Used to delete some data

Summary up until now...

- A backend listens to requests and responds to them over the internet
- The backend exposes some APIs to work with it
- The address isn't the only parameter to interact with the API
- HTTP methods are also included with each HTTP request
- REST paradigms can then be used to design APIs

Structured Data

- When we get a **request** for some **data**, we need to **send** it in a **structured** format
- One of the famous data formats nowadays is **JSON**
- **JSON** stands for **JavaScript Object Notation**
- It is a way to represent **key-value** pairs in JavaScript

Key value pairs

- **Key** is the **identifier** for the stored data
- **Value** is the **value** given to the identifier
- For example,

name : Mukesh

age: 22

Example JSON data

```
{  
  "id": 1,  
  "name": "Alice Johnson",  
  "age": 28,  
  "address": "123 Maple Street, Seattle, Washington, USA",  
  "hobbies": [ "Reading", "Hiking", "Photography" ]  
}
```

Python FastAPI

- Library in Python to create APIs easily
- Simple interface for handling different HTTP methods
- GET endpoint: `@app.get("/users")`
- POST endpoint: `@app.post("/users")`
- Handles JSON data by default

Using Postman

- Postman is a **tool** used to **analyse** API endpoints easily
- We can work with the various **HTTP methods**
- We can add **headers** and **body** easily
- The **request** and the **response** is easy to understand

Creating routes in FastAPI: GET endpoints

- Say we want to access the data of all users. Then the endpoint would be:

`localhost:8000/users`

- We can create a function that returns all the users inside this endpoint.

```
@app.get('/users')
```

```
def get_users():
```

```
    return users
```


Creating routes in FastAPI: Query parameters

- In GET request, the data sent to the server is seen in the URL like:

`localhost:8000/users?ageLimit=40`

- We can intercept the parameter easily in our server like:

```
@app.get('/users')
```

```
def users_with_age_limit(ageLimit: int):
```

```
    # Now, we can access the ageLimit variable inside the function
```

Creating routes in FastAPI: Query parameters

```
@app.get("/users")
def users_with_age_limit(ageLimit: Optional[int] = None):

    if ageLimit is None:
        return users

    filtered_users = [ user for user in users if user["age"] <= ageLimit ]

    return filtered_users
```

Creating routes in FastAPI: Path parameters

- Sometimes, the URL itself contains the data. It is called a **path parameter**.

`localhost:8000/users/10`

- We can get the parameter using resolution:

```
@app.get('/users{user_id}')
```

```
def get_user(user_id: int):
```

Now, we can access the user_id variable inside the function

Creating routes in FastAPI: Path parameters

```
@app.get("/users{user_id}")
def get_user(user_id: int):

    for user in users:
        if user["id"] == user_id:
            return user

    return {}
```

Creating routes in FastAPI: POST endpoints

- In FastAPI, we define the **structure** in which we expect the **payload** in the POST **request body**.

```
class User(BaseModel):  
    name: str  
    age: int  
    address: str  
    hobbies: List[str]
```

Creating routes in FastAPI: POST endpoints

```
@app.post("/users")
```

```
def add_user(user: User):
```

```
    new_user_id = len(users) + 1
```

```
    new_user = {
```

```
        "id": new_user_id,
```

```
        **user.dict()
```

```
    }
```

```
    users.append(new_user)
```

```
    return {
```

```
        "id": new_user_id
```

```
    }
```

Invoking API endpoints from JavaScript: Fetch API

- In **JavaScript**, the **fetch** API is used to send HTTP **requests** and work with the **responses**
- For **GET** request, the syntax is very simple:

```
const response = await fetch('url');  
const data = await response.json();
```

Invoking API endpoints from JavaScript: Fetch API

```
async function loadUsers(ageLimit){  
  
    let url = "/users";  
  
    if (ageLimit) {  
        url += `?ageLimit=${ageLimit}`;  
    }  
  
    const response = await fetch(url);  
  
    const users = await response.json();  
  
    // Uncomment remaining code
```


Invoking API endpoints from JavaScript: Fetch API

- For **POST** request, the syntax looks like this:

```
await fetch("/users",{  
    method: "POST",  
  
    headers: {  
        "Content-Type" : "application/json"  
    },  
  
    body: JSON.stringify(bodyObject)  
});
```

The End

Please go through the README for additional information